ASSIGNMENT 02

Object Oriented Programming with C++ (DSA0104)

Part 2: Questions Q4 to Q7

Name	Sangeet TR
Reg No	1921260245
Course Name	Object Oriented Programming with C++
Course Code	DSA0104
GitHub Repository	https://github.com/Sangeetthangaraj/DSA0104---object-oriented-programming-with-c-

Contents and marks

Q	Topic	Marks	Source folder
4	Specifying a class with an array inside it	6	Q4/
5	Static members and arrays of objects	7	Q5/
6	Friend functions and inline functions	8	Q6/
7	Diagnose, redesign and validate (transaction log)	13	Q7/

How the evidence was produced

- Every program was compiled with `g++ 13.3.0 -std=c++17 -Wall -Wextra` on Ubuntu 24.04 and executed. Compiler messages and program output in this report are copied verbatim from the files in `outputs/` of the GitHub repository (written by `run_all.sh`).
- Predictions were written from reasoning about the code before the program was run; each is compared with the observed result next to it.
- Timings come from a single-core virtual machine, so absolute times will differ on another machine. Only ratios between variants should be compared.
- This is Part 2 of 2. Part 1 covers Q1 to Q4. The source code of all programs is in the GitHub repository.

Q4 - Specifying a Class with an Array Inside It (6 marks)

4.1 Class specification

`SavingsAccount` has private data (account number, balance, an array of the last 10 transactions and a count) and public `deposit()`, `withdraw()` and `miniStatement()`. The getters and the constructor are defined **inside** the class; `deposit`, `withdraw`, `miniStatement` and the private helper `record` are defined **outside** with the scope resolution operator `SavingsAccount::`. A withdrawal larger than the balance is rejected and is not recorded.

q4_account.cpp

```
// Q4 - SavingsAccount with a private array of the last MAX_TXN transactions.
// Design 1: shifting array. Some members are defined inside the class,
// some outside with the scope resolution operator.
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };     // 'D' deposit, 'W' withdrawal

class SavingsAccount {
    static const int MAX_TXN = 10;       // change to 50 here and nowhere else
    int         accNo;
    double      balance;
    Transaction txn[MAX_TXN];            // private: no one can corrupt count/order
    int         count;                   // how many valid entries (0..MAX_TXN)

    void record(char type, double amount);         // helper, defined outside
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), count(0) {}

    int    getAccNo()   const { return accNo; }      // defined inside the class
    double getBalance() const { return balance; }

    void deposit(double amount);                     // defined outside
    bool withdraw(double amount);                    // defined outside
    void miniStatement() const;                      // defined outside
};

void SavingsAccount::record(char type, double amount)
{
    if (count == MAX_TXN) {                          // 11th transaction: shift left, O(k)
        for (int i = 1; i < MAX_TXN; ++i) txn[i - 1] = txn[i];
        --count;
    }
    txn[count].type = type;
    txn[count].amount = amount;
    ++count;
}
void SavingsAccount::deposit(double amount)
{
    if (amount <= 0) return;
    balance += amount;
    record('D', amount);
}
bool SavingsAccount::withdraw(double amount)
{
    if (amount <= 0 || amount > balance) return false;   // rejected: not recorded
    balance -= amount;
    record('W', amount);
    return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement for account " << accNo << " (last " << count << ")\n";
    for (int i = 0; i < count; ++i)
        std::cout << "  " << std::setw(2) << i + 1 << ". "
                  << (txn[i].type == 'D' ? "Deposit    " : "Withdrawal")
                  << std::setw(9) << txn[i].amount << '\n';
    std::cout << "  Balance: " << balance << '\n';
}

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; ++i) acc.deposit(100 * i);          // 8 transactions
```

```
    acc.withdraw(250);                                          // 9
    acc.withdraw(1e9);                                          // rejected
    acc.miniStatement();
    std::cout << "--- add 3 more (10th fills the array; 11th and 12th discard the two
oldest) ---\n";
    acc.deposit(1111); acc.withdraw(50); acc.deposit(2222);
    acc.miniStatement();
}
```

Captured output

```
Mini-statement for account 1001 (last 9)
   1. Deposit      100.00
   2. Deposit      200.00
   3. Deposit      300.00
   4. Deposit      400.00
   5. Deposit      500.00
   6. Deposit      600.00
   7. Deposit      700.00
   8. Deposit      800.00
   9. Withdrawal   250.00
  Balance: 8350.00
--- add 3 more (10th fills the array; 11th and 12th discard the two oldest) ---
Mini-statement for account 1001 (last 10)
   1. Deposit      300.00
   2. Deposit      400.00
   3. Deposit      500.00
   4. Deposit      600.00
   5. Deposit      700.00
   6. Deposit      800.00
   7. Withdrawal   250.00
   8. Deposit     1111.00
   9. Withdrawal    50.00
  10. Deposit     2222.00
  Balance: 11633.00
exit status: 0
```

4.2 Inquiry

Why the array must be private

The class has an **invariant**: `0 <= count <= MAX_TXN`, entries are in time order, and the balance equals the opening balance plus the recorded history. If `txn` and `count` were public, any code could overwrite a transaction, change `count` so that `miniStatement()` reads beyond the array, or change the balance without a record, bypassing the check in `withdraw`. Keeping the data private means only the three public functions can change it, and the **representation can be replaced** without touching any caller. Q7 does exactly that (shift array -> circular index) and the calling code in `main` does not change.

What happens when the 11th transaction arrives

	Shift the array	**Circular index**
Action	copy `txn[1..9]` to `txn[0..8]`, then write the new transaction at `txn[9]`	write at `(start + count) % 10`, then advance `start` by one (the oldest slot is overwritten)
Elements moved	9 (K - 1) on every add after the array is full	0
Cost per add	O(K)	O(1)
Statement order	physical order = time order, simple loop	physical order is rotated; read through `(start + i) % K`

Observed: after 12 transactions (the 9 shown first plus 3 more) the two oldest (deposits 100 and 200) were discarded and the list starts at 300. The shifting version, the circular version and the `std::deque` version printed **identical** statements (checked with `diff`; see 4.4). The measured cost of the two methods is in Q7.

4.3 Design Decision: the last 50 transactions

Decision: use the circular index. In both designs the capacity is the single constant `MAX_TXN`, so changing 10 to 50 is a one-line edit and no other code changes (loop bounds and the modulus are expressed with it). The difference is in what the change does to the running cost: with shifting, each add after the array is full now moves 49 elements instead of 9 (5.4 times more work); with the circular index the cost of an add stays one write. Memory grows in both from 160 bytes to 800 bytes per account (16 bytes per `Transaction` x 50), because an array always reserves its full size. The public interface, the callers and the tests are unchanged either way. If the ATM should print only part of the history, `miniStatement` can take a `lastN` parameter (done in `q7_redesign.cpp`).

4.4 Alternative concept and complexity analysis

Both alternatives keep the same public interface. The circular buffer:

q4_circular.cpp

```
// Q4 - Design 2: circular index (start + count). Same public interface.
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };

class SavingsAccount {
    static const int MAX_TXN = 10;
    int accNo; double balance;
    Transaction txn[MAX_TXN];
    int start;        // index of the OLDEST transaction
    int count;
    void record(char type, double amount)
    {
        int pos = (start + count) % MAX_TXN;           // where the newest goes
        txn[pos] = {type, amount};
        if (count < MAX_TXN) ++count;
        else start = (start + 1) % MAX_TXN;            // overwrite oldest: O(1)
    }
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), start(0), count(0)
{}
    void deposit(double a) { if (a <= 0) return; balance += a; record('D', a); }
    bool withdraw(double a) { if (a <= 0 || a > balance) return false; balance -= a;
record('W', a); return true; }
    void miniStatement() const
    {
        std::cout << "Mini-statement for account " << accNo << " (last " << count << ")\n";
        for (int i = 0; i < count; ++i) {
            const Transaction& t = txn[(start + i) % MAX_TXN];   // oldest -> newest
            std::cout << "  " << std::setw(2) << i + 1 << ". "
                      << (t.type == 'D' ? "Deposit   " : "Withdrawal")
                      << std::setw(9) << t.amount << '\n';
        }
        std::cout << "  Balance: " << balance << '\n';
    }
};

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    SavingsAccount acc(1001, 5000);
    for (int i = 1; i <= 8; ++i) acc.deposit(100 * i);
    acc.withdraw(250);
    acc.withdraw(1e9);
    acc.deposit(1111); acc.withdraw(50); acc.deposit(2222);
    acc.miniStatement();
}
```

The deque version replaces the array by `std::deque<Transaction>`: `record` becomes `if (txn.size() == MAX_TXN) txn.pop_front(); txn.push_back({t, a});` (full listing in `Q4/q4_deque.cpp`).

Captured output of q4_circular (identical to q4_deque; both equal the second statement of q4_account)

```
Mini-statement for account 1001 (last 10)
   1. Deposit      300.00
   2. Deposit      400.00
   3. Deposit      500.00
   4. Deposit      600.00
   5. Deposit      700.00
   6. Deposit      800.00
   7. Withdrawal   250.00
   8. Deposit     1111.00
   9. Withdrawal    50.00
  10. Deposit     2222.00
  Balance: 11633.00
exit status: 0
```

Design	Time to add	Space	Remarks
Shifting array	O(k): k = number of stored entries moved (K - 1 once full)	K x 16 bytes, plus `count`	simplest code; in-order storage
Circular buffer (start index)	O(1)	K x 16 bytes, plus `start` and `count`	fixed memory, no heap; index arithmetic
`std::deque` (push_back, pop_front)	O(1) amortised	up to K entries in heap blocks, plus bookkeeping (the deque object alone is 80 bytes on this compiler, before any element is stored)	not contiguous; heap allocation; capacity can grow without changing the class

The circular buffer has the best time and the fixed, predictable memory of an array. The deque is the most flexible but costs more per account in memory and allocations.

Q5 - Static Members and Arrays of Objects (7 marks)

5.1 From the principle to a class design

Principle: data that belongs to the whole class, not to one object, should exist only once however many objects are created. Applied to `BusTicket`:

Member	Belongs to	Kind
`issued` (number of seats given out)	the whole class (one count for the bus)	**static** data member
`CAPACITY` (number of seats)	the whole class (same for all tickets)	**static const** data member
`seat`, `name`	one particular ticket	ordinary (non-static) data members
`count()`	asks about the class total, no ticket needed	**static** member function

Where a static data member must be defined

The declaration `static int issued;` inside the class only **announces** the member; it does not create storage. Exactly **one** definition must appear outside the class in one source file: `int BusTicket::issued = 0;`. This is also where the initial value is given. If the class is in a header included by several `.cpp` files, the definition must stay in a single `.cpp`, otherwise the linker reports multiple definitions (or "undefined reference" when it is missing). (C++17 also allows `static inline int issued = 0;` inside the class.) A `static const int` of integral type, like `CAPACITY`, may be initialised inside the class.

Why a static member function cannot use this or a non-static member

A non-static member function is called on an object (`t.print()`) and receives a hidden `this` pointer to that object. A static member function is called on the class (`BusTicket::count()`) and receives **no** `this`. Because `seat` and `name` exist once per object, there is no object they could refer to, so the function can use only static members, its parameters and global names. The benefit is that it can be called when no object exists yet.

How an array of BusTicket objects is created and initialised

`BusTicket` has only a one-argument `explicit` constructor, so there is **no default constructor** and `BusTicket trip[4];` would not compile. Each element must be initialised in a braced list, one constructor call per element, in index order: `BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"), BusTicket("Chitra"), BusTicket("Dev")};`. The elements are constructed in order 0, 1, 2, 3 and destroyed in the reverse order. (With a default constructor, as in `Token counter[3];` below, the constructor runs once for every element.)

What happens to the count when tickets are created in a loop or a function

The static `issued` is **not tied to the lifetime of any ticket**. Tickets created in a loop or in a function are destroyed when they go out of scope, but nothing in the class decrements `issued`, so the count only ever goes up (to the capacity). This is the correct behaviour for "seats issued", but it means a cancelled ticket must be handled by an explicit `cancel()` function; a destructor cannot do it because a waiting-list ticket never took a seat. Also, a compiler-generated **copy** of a ticket does not run the constructor body, so it does not take a new seat but duplicates the seat number: copying should be disabled or controlled in a real system.

5.2 Predict the Output

q5_predict.cpp

```
// Q5 - Predict the Output (program given in the question, unchanged)
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
              << ' ' << Token::issued() << '\n';
}
```

My predicted output (written before running)

```
101 103 104 4
```

Reasoning

- The definition `int Token::next = 100;` gives the static member its initial value **before** `main` **starts**, so `next` is 100 when the first object is created.
- `Token counter[3];` runs the default constructor **three times**, for elements 0, 1 and 2: `++next` gives ids 101, 102, 103.
- `Token extra;` runs the constructor a **fourth** time: id 104.
- `Token::issued()` returns `next - 100` = 104 - 100 = 4.
- The output is `counter[0].get()` = 101, `counter[2].get()` = 103, `extra.get()` = 104, `issued()` = 4.

Observed output

```
101 103 104 4
exit status: 0
```

Differences between prediction and actual output: none. The static counter starts at 100 only because of its definition outside the class, and the constructor ran 4 times in total (3 for the array, 1 for `extra`), which is why `issued()` reports 4 even though no `Token` variable stores that number.

5.3 Complete the Code

q5_busticket.cpp

```
// Q5 - College Bus Seat Tickets (TODO 1 and TODO 2 completed)
#include <iostream>
#include <string>

class BusTicket {
    static int issued;                  // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1 (done)
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;              // next seat number
        }                               // else seat stays 0 -> waiting list
    }
    static int count()
    {
        // TODO 2 (done)
        return issued;
```

```
    }
    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
};
int BusTicket::issued = 0;              // definition of the static data member

void bookBatch(const char* label)
{
    std::cout << "-- " << label << " --\n";
    BusTicket local[2] = {BusTicket("Eswar"), BusTicket("Fathima")};
    for (const BusTicket& t : local) t.print();
    std::cout << "Issued inside function: " << BusTicket::count() << '\n';
}   // the two tickets are destroyed here, but 'issued' is not reduced

int main()
{
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\n';

    bookBatch("function call 1");
    bookBatch("function call 2");
    std::cout << "Issued after both calls: " << BusTicket::count() << '\n';
}
```

The `main` function of the question gives the expected output below. The program `q5_expected.cpp` is the same class with exactly that `main`:

Observed output of the question's main (matches the expected output: Seat 1, Seat 2, Seat 3, Waiting, Issued: 3)

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
exit status: 0
```

Array larger than the capacity, and tickets created inside a function called twice

The array of 4 tickets with a capacity of 3 gives seats 1, 2, 3 and puts Dev on the waiting list. `bookBatch` then creates two more tickets in a function and is called twice:

Captured output of q5_busticket

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
-- function call 1 --
Eswar: Waiting
Fathima: Waiting
Issued inside function: 3
-- function call 2 --
Eswar: Waiting
Fathima: Waiting
Issued inside function: 3
Issued after both calls: 3
exit status: 0
```

The count stays **3** during and after both calls: the seats were already taken by the array in `main`, so both new tickets go to the waiting list, and the destruction of the local tickets at the end of each call does not reduce `issued`. If `bookBatch` were called before the array was created, its tickets would take seats 1 and 2 and those seats would stay issued after the function returned.

5.4 Alternative concept and complexity analysis

Alternative: a `TicketOffice` object per bus, passed to every ticket (`q5_alt_office.cpp`). Two buses of capacity 2 run at once:

Captured output of q5_alt_office

```
Asha: Seat 1
Bala: Seat 2
Chitra: Waiting
Dev: Seat 1
bus1 issued 2, bus2 issued 1
exit status: 0
```

Design	Time per ticket	Space	Encapsulation	Two buses at once
Global variable	O(1)	one counter	poor: any code can change it; name clashes; initialisation order problems between source files	No (needs two globals and different code)
Static data member	O(1)	one counter for the whole program	good: private, visible only to the class	No: one counter per class, so one bus
`TicketOffice` object passed to each ticket	O(1)	one counter per bus (each ticket only needs it while being constructed)	good: the counter belongs to the office object	**Yes**, as the output shows: bus 1 issued 2, bus 2 issued 1

Time is O(1) in all three. The static member is the right answer to the question as asked (one bus, one class-wide count, no extra arguments). When the requirement changes to several buses, the `TicketOffice` design is better, because the counter then depends on the bus and not on the class.

Q6 - Friend Functions and Inline Functions (8 marks)

6.1 Design of the comparison

Why a friend function is needed when the function works on two objects equally

`minutesBetween(a, b)` and `earlier(a, b)` treat both times equally. A **member** function has a special left operand: `a.minutesTo(b)` makes `a` the object and `b` an argument, which suggests a difference that does not exist, and the left operand is not subject to implicit conversions. A **non-member** function treats both as ordinary parameters. But a non-member cannot read the private `h` and `m`. Declaring it a `friend` gives that one function (and no other code) access to the private data without adding public getters.

How a friend function differs from a member function

	Member function	**Friend function**
Call	`a.minutesTo(b)`	`minutesBetween(a, b)`
`this` pointer	yes (hidden, points to `a`)	no (both objects are explicit parameters)
Access to private members	of its own object (and of other objects of the class)	of every object of the class it receives, as granted by the class
Part of the class?	yes (scope `Time::`)	no; only declared in the class; the access specifier where it is declared does not matter
Inherited / overridden?	yes, can be virtual	no; friendship is not inherited and not mutual

Should the small functions be inline, and what does inline ask the compiler to do?

`inline` asks the compiler to **substitute the function body at the call** instead of generating a call (saving the call and return and enabling further optimisation). It is a request, not a command: the compiler may refuse, and an optimising compiler may also inline functions that are **not** marked `inline`. Modern C++ also gives `inline` a linkage meaning: the function may be defined in several translation units (as when it is in a header) without breaking the one-definition rule. These functions are tiny (one subtraction, one comparison, a few multiplications), so they are good candidates. The measurements in 6.4 show how much it matters.

What would be lost if the private members were made public

- **Validity is lost:** nothing stops `t.m = 75` or `t.h = -3`, so every function would have to defend itself.
- **Freedom to change the representation is lost:** if `Time` later stored minutes since midnight, every piece of code that used `h` and `m` would have to be rewritten.
- Debugging becomes harder because any code in the program could have corrupted a time. Friends keep the number of places that can see `h` and `m` to two known functions.

A limitation of the given `Time`: its constructor does not validate its arguments (`Time(25, 70)` is accepted); a real class should check them in the constructor.

6.2 Complete the Code

q6_time.cpp

```
// Q6 - Time class with friend functions (TODO 1 and TODO 2 completed) + tests.
#include <iostream>
#include <cstdlib>

class Time {
```

```
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1 (done): absolute difference in minutes, uses private h and m
    return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));
}
bool earlier(const Time& a, const Time& b)
{
    // TODO 2 (done): true if a is earlier in the day than b
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main()
{
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha << earlier(arrival, departure) << '\n';

    std::cout << "--- correctness checks ---\n";
    Time t1(10, 30), t2(10, 30);                        // same time twice
    std::cout << "same time      : " << minutesBetween(t1, t2) << ' ' << earlier(t1, t2)
              << "  (hand: 0 false)\n";
    Time a(11, 50), b(12, 10);                          // either side of noon
    std::cout << "11:50 vs 12:10 : " << minutesBetween(a, b) << ' ' << earlier(a, b)
              << "  (hand: 20 true)\n";
    std::cout << "12:10 vs 11:50 : " << minutesBetween(b, a) << ' ' << earlier(b, a)
              << "  (hand: 20 false)\n";
    Time c(0, 0), d(23, 59);
    std::cout << "00:00 vs 23:59 : " << minutesBetween(c, d) << ' ' << earlier(c, d)
              << "  (hand: 1439 true)\n";
    Time e(12, 0), f(11, 59);
    std::cout << "12:00 vs 11:59 : " << minutesBetween(e, f) << ' ' << earlier(e, f)
              << "  (hand: 1 false)\n";
}
```

Captured output (first line is the expected output: 205 false)

```
205 false
--- correctness checks ---
same time      : 0 false  (hand: 0 false)
11:50 vs 12:10 : 20 true  (hand: 20 true)
12:10 vs 11:50 : 20 false  (hand: 20 false)
00:00 vs 23:59 : 1439 true  (hand: 1439 true)
12:00 vs 11:59 : 1 false  (hand: 1 false)
exit status: 0
```

6.3 Correctness check

Verified by hand before running: 9:45 = 585 minutes since midnight, 13:10 = 790, difference 205; arrival is not earlier than departure, so `false`.

Case	Hand calculation	Expected	Observed
10:30 and 10:30 (same time twice)	630 - 630 = 0; 630 < 630 is false	0 false	0 false
11:50 and 12:10 (either side of noon)	710 and 730: difference 20; 710 < 730	20 true	20 true
12:10 and 11:50 (arguments swapped)	difference is absolute, so 20; 730 < 710 is false	20 false	20 false
00:00 and 23:59 (extremes)	0 and 1439	1439 true	1439 true
12:00 and 11:59 (one minute apart)	720 and 719	1 false	1 false

Limitation found while testing: `minutesBetween` is an absolute difference within one day. A train leaving at 23:30 and arriving at 01:10 the next day would give 1340 minutes instead of 100. A real

journey planner needs a day offset (for example a `Time` that includes a day number, or arrival minutes + 1440 when arrival is earlier than departure).

6.4 Growth investigation: 10,000,000 calls with and without inline

`q6_bench.cpp` calls `minutesBetween` and `earlier` 10,000,000 times on 1024 pseudo-random times, five repetitions per variant, median reported. "friend, noinline" is the same code with `__attribute__((noinline))`, which forces a real call even when optimising. Member functions and public getters are the alternatives asked in the next section. All variants give the same checksum (4716798628), so they compute the same result.

Variant (10,000,000 calls of both functions)	-O0 (ms)	-O2 (ms)	-O0 / -O2
friend, inline	74.8	15.3	4.9x
friend, noinline	74.0	20.7	3.6x
member functions	71.1	15.6	4.6x
public getters	139.4	15.4	9.0x

Five separate executions at `-O2` gave 15.3 to 18.6 ms for the inlined friend functions and 20.7 to 22.6 ms for the non-inlined version, so the ordering in the table is stable although individual numbers move by about 10%.

Explanation of the result

- **At -O0 nothing is inlined**, even for functions marked `inline`: the compiler does no optimisation, so "friend, inline" and "friend, noinline" take the same time (about 74 ms). The keyword by itself did not make the program faster.
- **At -O0 the getters are about 1.9 times slower** than the friend and member versions: each comparison makes four extra real function calls (`getH`, `getM` twice). Encapsulation through getters costs something only when the compiler does not inline.
- **At -O2 the optimiser inlines the small functions** and the whole loop becomes about 4.9 times faster than at -O0. Friend, member and getter designs then take about the same time (15 to 16 ms), so a getter-based design is free in an optimised build.
- **At -O2, forbidding inlining costs about 35% more** (20.7 ms against 15.3 ms): about 0.5 ns per pair of calls. This is the true effect of `inline`. In absolute terms ten million calls cost 15 to 21 ms, so the planner is unlikely to be limited by call overhead.

6.5 Deployment Decision

Decision: keep `minutesBetween` and `earlier` as friend functions, define them inline in the header, and do not add public getters. Reasons:

- Both functions are symmetric operations on two objects, which is what a non-member friend expresses naturally.
- Friendship exposes `h` and `m` to exactly two functions; public getters would expose the representation to the entire program and would be 1.9 times slower in a debug (-O0) build.
- The measurements show no penalty at `-O2` for friends compared with members or getters, so the choice can be made on design grounds.
- Replacing them by member functions (`a.minutesTo(b)`, `a.isEarlier(b)`) would also be acceptable, with identical speed (15.6 ms at -O2); I would switch to members or to `operator<` if the class gains more comparison operators or subclasses.

6.6 Alternative concept and complexity analysis

Design	Call	Time per call	Measured -O0 / -O2 (ms per 10 M calls)	Encapsulation
Friend functions	`minutesBetween(a, b)`	O(1)	74.8 / 15.3	good: two named functions see the private data
Member functions	`a.minutesTo(b)`	O(1)	71.1 / 15.6	good: no friend needed; asymmetric syntax
Public getters + ordinary functions	`minutesGetters(a, b)`	O(1)	139.4 / 15.4	weakest: the representation (hours and minutes) becomes part of the public interface

After inlining the three designs are the same machine code in effect, so the time per call is the same O(1) (a few arithmetic instructions); the choice is about encapsulation and clarity. The space per call is two references (no object copied) in all three designs.

Captured benchmark output at -O0

```
10,000,000 calls of minutesBetween + earlier per variant
friend, inline          : 74.8465 ms (median of 5), checksum 4716798628
friend, noinline        : 74.0246 ms (median of 5), checksum 4716798628
member functions        : 71.1396 ms (median of 5), checksum 4716798628
public getters          : 139.428 ms (median of 5), checksum 4716798628
exit status: 0
```

Captured benchmark output at -O2

```
10,000,000 calls of minutesBetween + earlier per variant
friend, inline          : 15.3315 ms (median of 5), checksum 4716798628
friend, noinline        : 20.7228 ms (median of 5), checksum 4716798628
member functions        : 15.6084 ms (median of 5), checksum 4716798628
public getters          : 15.4397 ms (median of 5), checksum 4716798628
exit status: 0
```

Q7 - Diagnose, Redesign and Validate (13 marks)

Program selected: the `SavingsAccount` **transaction log of Q4** (shifting array), because the Q4 requirement changed from 10 to 50 transactions and the Q4 analysis predicts a cost that depends on the capacity. The comparison program is `Q7/q7_bench.cpp`; the redesigned class is `Q7/q7_redesign.cpp`.

A. Diagnose the limitation (2 marks)

In the original design, `record()` moves K - 1 entries to the left on **every** transaction once the array is full. I measured 100,000 transactions added to a log of capacity K, original design (median of 3 runs, `-O2`):

Capacity K	Total time (ms)	Time per add (ns)	Growth against K = 10
10	0.80	8	1.0x
50	1.46	15	1.8x
500	5.95	60	7.4x
5000	154.65	1547	193.3x

The time per add grows with K (about 8 ns at K = 10, 15 ns at K = 50, 60 ns at K = 500, 1550 ns at K = 5000): going from the current 10 to the requested 50 transactions already costs about 1.8 times more per add, and 5000 is about 190 times slower. The growth between 500 and 5000 (26 times for 10 times more entries) is larger than linear because the array (5000 x 16 bytes = 80 KB) no longer fits in the fastest cache.

Cause in the original design: the array is kept in time order **in physical positions**: the oldest entry must always be at index 0. Removing the oldest entry therefore forces every other entry to move, which makes the cost of one add O(K) although only one entry is added and one is dropped.

B. Design objective (2 marks)

Objective (measurable): make the cost of recording a transaction **independent of the capacity K**, while keeping the same stored content, the same public behaviour and no more than 4 extra bytes per account. The redesign succeeds if, with 100,000 adds at -O2:

1. the time at K = 5000 is **no more than 1.5 times** the time at K = 10 for the same design (flat cost);
2. the redesign is **at least 10 times faster** than the original at K = 500;
3. the stored transactions are **identical** to the original for every K tested.

C. Design the revised solution (4 marks)

What changes. The array stays, but "oldest at index 0" is dropped as a rule. A new private member `start` holds the index of the oldest entry. A new transaction is written at the physical position `(start + count) % K`; when the log is full that position is exactly the oldest entry, which is overwritten, and `start` moves one step forward. Logical entry `i` (0 = oldest) is at physical index `(start + i) % K`.
C++ features used: private data members and encapsulation (callers cannot see the change), a member function defined inside the class, a `const` member accessor `at(i)` for logical access, and modulo arithmetic; for the benchmark a non-type template parameter `K` makes both designs use compile-time capacities.

Class diagram (text)

```
+-------------------------------------------+
|  SavingsAccount                           |
+-------------------------------------------+
| - accNo : int                             |
| - balance : double                        |
| - txn[MAX_TXN] : Transaction  (fixed)     |
| - start : int      <-- NEW (oldest entry) |
| - count : int                             |
+-------------------------------------------+
| + deposit(amount)                         |
| + withdraw(amount) : bool                 |
| + miniStatement(lastN)                    |
| - record(type, amount)   <-- changed      |
+-------------------------------------------+
```

Pseudocode of the changed function

```
record(type, amount):
    pos = (start + count) mod K
    txn[pos] = (type, amount)
    if count < K:
        count = count + 1            // still filling
    else:
        start = (start + 1) mod K    // overwrite oldest, O(1)

entry(i):  return txn[(start + i) mod K]     // i = 0 is the oldest
```

Extra memory: one `int` (`start`, 4 bytes) per account; the array itself is unchanged. **New trade-offs:** (1) every read of a logical entry pays an addition and a wrap test; (2) the array is no longer in time order, so it cannot be copied or written to a file as it stands; (3) off-by-one errors in the wrap-around are easy to make, which is why the benchmark checks the contents against the original; (4) the capacity is still fixed and every account reserves its full array.

D. Predict before implementing (1 mark)

Prediction made from the algorithm before running the benchmark: the original design does K - 1 element moves per add and the redesign does one write, so the **ratio original / redesign grows with K**. I expect the redesign time to be flat (within about 50% of itself) for every K, a speed-up of **at least 5 times at K = 50** and **at least 20 times at K = 500**, and a very large speed-up at K = 5000. The measure is the total time for 100,000 adds. At K = 10 (9 moves) I expect only a small gain (roughly 2 to 4 times).

E. Implement and validate (2 marks)

The benchmark uses the same sequence of 100,000 transactions for both designs and, after each run, compares a checksum over the stored entries in logical order. The same tests (Q4 statements) were also run on the real classes and the output compared with `diff`: shifting, circular and deque produce identical statements.

Original -> Redesigned at -O2

K	Original: shift (ms)	Redesigned: ring (ms)	Speed-up	Stored content
10	0.80	0.27	3.0x	identical
50	1.46	0.18	8.1x	identical
500	5.95	0.24	24.4x	identical
5000	154.65	0.18	864.5x	identical

Original -> Redesigned at -O0 (no optimisation)

K	Original: shift (ms)	Redesigned: ring (ms)	Speed-up	Stored content
10	2.10	0.55	3.8x	identical

K	Original: shift (ms)	Redesigned: ring (ms)	Speed-up	Stored content
50	8.26	0.63	13.2x	identical
500	63.28	0.47	133.5x	identical
5000	667.66	0.61	1092.4x	identical

Captured benchmark output at -O2

```
Adding N = 100000 transactions; time in ms (median of 3)
     K  shift (orig)    ring (new)   speed-up  content
    10          0.80          0.27       3.0x      same
    50          1.46          0.18       8.1x      same
   500          5.95          0.24      24.4x      same
  5000        154.65          0.18     864.5x      same
exit status: 0
```

Redesigned class with a capacity of 50: 60 transactions are added (every third one a withdrawal), only the last 50 are kept, and the five newest are printed. Checked by hand: transactions 56 to 60 are deposit 560, withdrawal 57, deposit 580, deposit 590, withdrawal 60, which is exactly what is printed.

q7_redesign.cpp

```
// Q7 - The redesigned SavingsAccount (circular index) with a capacity of 50.
// Only ONE constant (MAX_TXN) differs from the 10-transaction version.
#include <iostream>
#include <iomanip>

struct Transaction { char type; double amount; };

class SavingsAccount {
    static const int MAX_TXN = 50;
    int accNo; double balance;
    Transaction txn[MAX_TXN];
    int start;                  // index of OLDEST transaction
    int count;                  // number stored (0..MAX_TXN)
    void record(char type, double amount)
    {
        int pos = start + count; if (pos >= MAX_TXN) pos -= MAX_TXN;
        txn[pos] = {type, amount};
        if (count < MAX_TXN) ++count;
        else if (++start == MAX_TXN) start = 0;             // drop oldest in O(1)
    }
public:
    SavingsAccount(int no, double opening) : accNo(no), balance(opening), start(0), count(0)
{}
    void deposit(double a)  { if (a <= 0) return; balance += a; record('D', a); }
    bool withdraw(double a) { if (a <= 0 || a > balance) return false; balance -= a;
record('W', a); return true; }
    int stored() const { return count; }
    void miniStatement(int lastN) const                      // print newest lastN, oldest
first
    {
        if (lastN > count) lastN = count;
        std::cout << "Account " << accNo << ": showing last " << lastN << " of " << count <<
" stored\n";
        for (int i = count - lastN; i < count; ++i) {
            int p = start + i; if (p >= MAX_TXN) p -= MAX_TXN;
            std::cout << "  " << (txn[p].type == 'D' ? "Deposit   " : "Withdrawal")
                      << std::setw(9) << txn[p].amount << '\n';
        }
    }
};

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    SavingsAccount acc(1001, 100000);
    for (int i = 1; i <= 60; ++i) {                          // 60 transactions
        if (i % 3) acc.deposit(i * 10); else acc.withdraw(i);
    }
    std::cout << "stored after 60 transactions: " << acc.stored() << " (capacity 50)\n";
    acc.miniStatement(5);
}
```

Captured output of q7_redesign

```
stored after 60 transactions: 50 (capacity 50)
Account 1001: showing last 5 of 50 stored
  Deposit      560.00
  Withdrawal    57.00
  Deposit      580.00
  Deposit      590.00
  Withdrawal    60.00
exit status: 0
```

F. Defend the final design (2 marks)

Was the objective achieved?

Objective	Evidence	Met?
Redesign flat: time at K = 5000 <= 1.5 x time at K = 10	0.18 ms at K = 5000 and 0.27 ms at K = 10 (-O2); 0.61 and 0.55 ms at -O0	Yes (flat; the K = 10 value is at timer noise level)
At least 10 times faster than the original at K = 500	24.4x at -O2, 133.5x at -O0	Yes
Identical stored content	"identical" for all K at both optimisation levels; Q4 statements identical by diff	Yes
At most 4 extra bytes per account	one `int` (`start`)	Yes

Prediction check. Predicted at least 5 times at K = 50 (measured 8.1x) and at least 20 times at K = 500 (measured 24.4x): both held. The gain at K = 10 was 3.0x, inside the 2 to 4 times predicted. The K = 5000 gain (864.5x) was larger than a simple count of moves suggests, because the shifting version also streams 80 KB through the cache for every add.

New costs. About 4 bytes more per account, a wrap-around calculation on every read, a physical order that is no longer chronological, and code that is harder to read and to get right. The unit tests of the mini-statement order are now more important than before.

Where the original may still be preferable. For the real requirement the original is not a problem: at K = 10 a shift moves 9 entries of 16 bytes, about 8 ns, and a single account receives a handful of transactions per day, so the saving of the redesign is a few nanoseconds per transaction. The original stores the entries in time order, so the statement is a plain loop and the array can be copied or saved directly; it is the better choice for K = 10 or 50 in a program where simplicity and ease of checking matter more than speed. The redesign is justified when K is large (hundreds or thousands), when many transactions are added in a batch (for example replaying a statement file), or when the same log class is reused for high-rate data. `std::deque` gives the same O(1) behaviour with simpler code but allocates heap memory per account (the deque object alone is 80 bytes before storing anything), so for a fixed small K the circular array remains the most compact.

Conclusion. The redesign meets its objective: the cost of an add no longer depends on K, at the price of one extra `int` and more intricate index arithmetic. For the immediate requirement (10, later 50 transactions) I would adopt it because it removes the dependency on K at almost no cost, but I would not claim it is needed at those sizes.